AhamX Bodhi

Where I Becomes Infinite

SentencePiece

Module 3.3: Tokenization

Prof. Sashikumaar Ganesan | IISc Bengaluru

Session Overview

What You Will Learn

- Why language-agnostic tokenization matters
- SentencePiece's two algorithms: BPE and Unigram
- How SentencePiece avoids pre-tokenization
- The Unigram Language Model algorithm
- SentencePiece in T5, Llama 2, Gemma, and NLLB

Concept at a Glance

- **Duration:** 8 minutes
- **Bloom's Level:** B3 (Apply)
- **Difficulty:** L2 (Intermediate)
- **Key Idea:** Language-agnostic, raw text tokenization
- **Used By:** T5, Llama 2, Gemma, mT5, NLLB

Learning Objectives

By the End of This Concept You Will Be Able To

- **Explain** why SentencePiece was designed as a language-agnostic tokenizer
- **Describe** the key difference: SentencePiece operates on raw text without language-specific pre-tokenization
- **Contrast** the BPE mode vs the Unigram Language Model mode in SentencePiece
- **Interpret** the whitespace-as-underscore convention used by SentencePiece
- **Identify** major multilingual and generative models that rely on SentencePiece

The Pre-Tokenization Problem

BPE and WordPiece Assume Whitespace Word Boundaries

- Both BPE and WordPiece split text on whitespace **before** applying their algorithms
- This works for English, French, German — languages that use spaces between words
- It fails for **Japanese, Chinese, Korean, Thai**: these languages have no whitespace word boundaries
- It also fails for agglutinative languages like Finnish and Turkish with extremely long compound words

SentencePiece (Kudo and Richardson, Google, 2018) was designed to remove this assumption entirely, making subword tokenization work **uniformly across all human languages**.

$$\frac{1}{Re}\Delta u_i - \nabla p + u \cdot \nabla u_i = 0$$

SentencePiece's Key Innovation

Raw Text Input, No Pre-Tokenization

- SentencePiece treats the entire input sentence as a **sequence of Unicode characters**, including spaces
- Whitespace is treated as a regular character and encoded as **underscore** (_) in the vocabulary
- No language-specific word segmenter is required as a pre-processing step
- The same model, trained on raw text, works for English, Japanese, Arabic, and any other language
- This makes SentencePiece a **self-contained, portable** tokenizer

The Underscore Convention

How Whitespace Becomes Part of the Token

- Whitespace before a word is encoded into the token as a leading _ (underscore)
- Example: ``Hello world'' → [_Hello, _world]
- Continuation subwords have **no** leading underscore: ``tokenization'' → [_token, ization]
- This is the **opposite** convention from WordPiece's ## suffix
- Decoding: replace _ with a space and concatenate all tokens

SentencePiece: BPE Mode

BPE Without Pre-Tokenization

- SentencePiece can run the standard BPE algorithm on raw character sequences
- No word boundary assumptions — spaces are just another character to merge
- Produces the same type of merge table as standard BPE
- Llama 2, Mistral, and Falcon use SentencePiece in BPE mode
- Vocabulary sizes: typically 32,000–64,000 tokens

The BPE mode in SentencePiece provides a drop-in replacement for language-specific BPE, with the key advantage that it works without pre-written word segmenters for each target language.

The Unigram Language Model Algorithm

SentencePiece's Second Algorithm

- The Unigram algorithm is the **other** option in SentencePiece (Kudo, 2018)
- Start with a very large vocabulary (e.g., all substrings up to length 16)
- Iteratively **prune** tokens whose removal least decreases training data likelihood
- Uses the Expectation-Maximization (EM) algorithm to estimate token probabilities
- Continue pruning until the target vocabulary size is reached
- Result: a probabilistic tokenizer that can produce **multiple valid segmentations** of the same word

Unigram vs BPE: Growth vs Pruning

BPE: Bottom-Up Growth

- Start small (characters only)
- Grow vocabulary by merging pairs
- Deterministic: one segmentation per word
- Merge order encodes the vocabulary

Unigram: Top-Down Pruning

- Start large (all candidate subwords)
- Shrink vocabulary by removing tokens
- Probabilistic: multiple segmentations possible
- Log-probabilities stored for each token

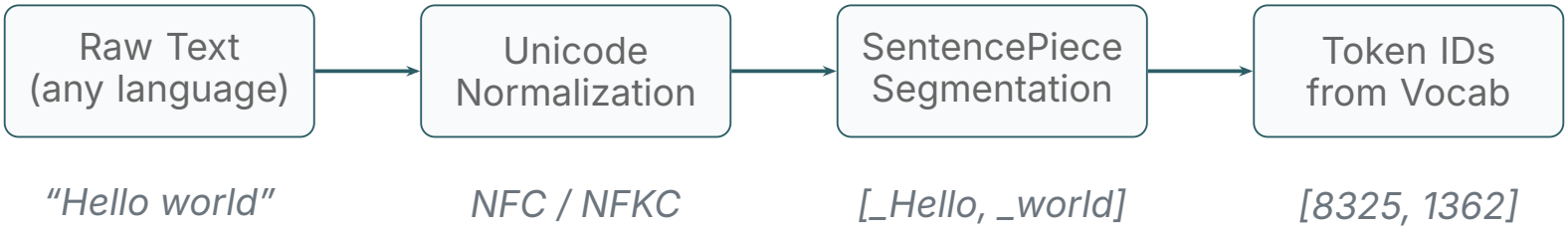
Unigram: Probabilistic Segmentation

Multiple Valid Tokenizations

- The Unigram model assigns a probability to every possible segmentation of a string
- ``New York'' could tokenize as `[_New, _York]`, `[_New, _Yor, k]`, or `[_N, ew, _Y, ork]`
- At inference, the **most probable** (Viterbi) segmentation is selected
- During training, **sampling** from multiple segmentations acts as data augmentation
- This regularization helps the model generalize to unseen morphological forms

The sampling capability of Unigram is its main advantage over BPE: the same word can be tokenized differently across training examples, preventing the model from over-fitting to specific token boundaries.

SentencePiece Pipeline



SentencePiece includes Unicode normalization (NFC or NFKC) as a built-in step, ensuring consistent handling of accented characters, ligatures, and composed/decomposed Unicode forms across all languages.

GenAI Application: T5 and mT5

SentencePiece in Google's T5 Family

- T5 (Text-to-Text Transfer Transformer, 2020) uses SentencePiece Unigram with 32,000 tokens
- All tasks are framed as text-in, text-out: the same tokenizer handles both input and output
- mT5 (multilingual T5) extends to 101 languages using the same SentencePiece framework with 250,000 tokens
- The Unigram algorithm's probabilistic sampling improved mT5's cross-lingual transfer

T5's success demonstrated that a single SentencePiece tokenizer, shared between encoder and decoder, is sufficient for state-of-the-art performance across summarization, translation, QA, and classification tasks.

GenAI Application: Llama 2 and Google Gemma

Open-Source Generative Models with SentencePiece

- **Llama 2 (Meta, 2023):** SentencePiece BPE, 32,000 token vocabulary. Fine-tunable on any hardware; widely used in open-source applications
- **Gemma (Google DeepMind, 2024):** SentencePiece Unigram, 256,128 vocabulary. Designed for deployment on edge devices and mobile
- **Gemma 2:** Extended SentencePiece vocabulary, improved multilingual coverage
- Both use byte-level fallback in SentencePiece to handle any Unicode character

GenAI Application: NLLB and Multilingual Translation

No Language Left Behind (Meta, 2022)

- NLLB is Meta's model covering 200+ languages, designed to translate between any pair
- Uses a SentencePiece Unigram tokenizer with 256,206 token vocabulary
- The large vocabulary ensures sufficient coverage for low-resource languages
- Language identity tokens (e.g., `__hin_Deva__`) are injected as special tokens
- SentencePiece's language-agnostic design was essential — no language-specific tools for 200+ languages

NLLB demonstrates SentencePiece's core design goal: one tokenizer that works for every human language, from resource-rich English to extremely low-resource languages like Luganda.

GenAI Application: Using SentencePiece in Code

SentencePiece Python API

- Install: `pip install sentencepiece`
- Train: `spm.SentencePieceTrainer.train(input=..., vocab_size=32000, model_type=`bpe')`
- Load: `sp = spm.SentencePieceProcessor(model_file=`my.model')`
- Encode: `sp.encode(`Hello world', out_type=str)` → `[`_Hello', `_world']`
- Encode to IDs: `sp.encode(`Hello world')` → `[8325, 1362]`
- Decode: `sp.decode([8325, 1362])` → `` `Hello world' ``

Three Tokenizers: Summary Comparison

Property	BPE	WordPiece	SentencePiece
Pre-tokenization	Whitespace split	Whitespace split	None (raw text)
Algorithm	Frequency merge	Likelihood merge	BPE or Unigram
Languages	Latin-script best	Latin-script best	All languages
Subword marker	Space prefix (`''`)	## continuation	_ word start
Inference	Replay merge rules	Longest-match-first	Viterbi decode
Probabilistic	No	No	Yes (Unigram)
Key Models	GPT-4, Llama 3	BERT, Electra	T5, Llama 2, Gemma

Common Misconceptions About SentencePiece

Myths and Corrections

- **Myth:** “SentencePiece is a single algorithm.” It is a *library* implementing both BPE and Unigram
- **Myth:** “The _ prefix means a space token.” It marks the *start* of a new word, encoding the space before that word
- **Myth:** “SentencePiece is only for non-English models.” T5, Llama 2, and Gemma — English-dominant models — all use it
- **Myth:** “Unigram is always better than BPE.” Unigram’s probabilistic sampling helps during training but the deterministic (Viterbi) mode is used at inference like BPE

Key Takeaways

What You Have Learned

- SentencePiece operates on raw text without language-specific pre-tokenization, making it universal
- Whitespace is encoded as a leading _ on word-starting tokens
- It supports two algorithms: **BPE** (deterministic merge) and **Unigram LM** (probabilistic pruning)
- The Unigram algorithm enables **tokenization sampling** during training, acting as regularization
- SentencePiece powers T5, Llama 2, Gemma, NLLB, and hundreds of multilingual models
- The library provides a self-contained, portable C++ implementation with Python bindings

Thank You

SentencePiece — Module 3.3